

```
import network
import socket
import time
import gc
from machine import Pin, PWM, I2C, UART
```

```
from vl53l0x import VL53L0X
from mpu6500 import MPU6500
```

```
# =====
```

```
# VEHICLE 2 - COMPLETE SYSTEM
```

```
# =====
```

```
#
```

```
# Sensors:
```

```
# VL53L0X
```

```
# HC-SR04
```

```
# MPU6500
```

```
# NEO-6M GPS
```

```
#
```

```
# Communication:
```

```
# Wi-Fi V2V UDP
```

```
#
```

```
# Actuators:
```

```
# TB6612FNG + 2 motors
```

```
# Buzzer
```

#

=====

=====

WIFI / V2V SETTINGS

=====

SSID = "Nothing Phone (4a)_Shreyas"

PASSWORD = "12345678"

UDP_PORT = 5005

Vehicle 1 IP

LEADER_IP = "10.63.66.108"

=====

MOTOR DRIVER - TB6612FNG

=====

AIN1 = Pin(2, Pin.OUT)

AIN2 = Pin(3, Pin.OUT)

PWMA = PWM(Pin(4))

BIN1 = Pin(5, Pin.OUT)

```
BIN2 = Pin(18, Pin.OUT)
```

```
PWMB = PWM(Pin(6))
```

```
STBY = Pin(7, Pin.OUT)
```

```
PWMA.freq(1000)
```

```
PWMB.freq(1000)
```

```
STBY.value(1)
```

```
# =====
```

```
# BUZZER
```

```
# =====
```

```
BUZZER = Pin(20, Pin.OUT)
```

```
BUZZER.value(0)
```

```
# =====
```

```
# HC-SR04
```

```
# =====
```

```
TRIG = Pin(13, Pin.OUT)
```

```
ECHO = Pin(19, Pin.IN)
```

TRIG.value(0)

=====

I2C

VL53L0X + MPU6500

=====

i2c = I2C(

0,

scl=Pin(1),

sda=Pin(0),

freq=400000

)

=====

GPS - NEO-6M

=====

gps = UART(

1,

baudrate=9600,

tx=Pin(8),

rx=Pin(9)

)

```
# =====
```

```
# GLOBAL VARIABLES
```

```
# =====
```

```
leader_speed = 0
```

```
leader_distance = 400.0
```

```
leader_status = "UNKNOWN"
```

```
leader_gps = "NO FIX"
```

```
last_v2v_time = 0
```

```
gps_lat = None
```

```
gps_lon = None
```

```
gps_fix = False
```

```
gps_satellites = 0
```

```
# =====
```

```
# MOTOR FUNCTIONS
```

```
# =====
```

```
def set_motor(motor_pwm, in1, in2, speed):
```

```
    if speed <= 0:
```

```
in1.value(0)
```

```
in2.value(0)
```

```
motor_pwm.duty_u16(0)
```

```
else:
```

```
in1.value(1)
```

```
in2.value(0)
```

```
duty = int((speed / 100) * 65535)
```

```
if duty > 65535:
```

```
    duty = 65535
```

```
motor_pwm.duty_u16(duty)
```

```
def motor_forward(speed):
```

```
    if speed < 0:
```

```
        speed = 0
```

```
    if speed > 100:
```

```
        speed = 100
```

```
STBY.value(1)
```

```
set_motor(  
    PWMA,  
    AIN1,  
    AIN2,  
    speed  
)
```

```
set_motor(  
    PWMB,  
    BIN1,  
    BIN2,  
    speed  
)
```

```
def motor_stop():
```

```
    PWMA.duty_u16(0)
```

```
    PWMB.duty_u16(0)
```

```
    AIN1.value(0)
```

```
    AIN2.value(0)
```

```
    BIN1.value(0)
```

```
BIN2.value(0)
```

```
# =====
```

```
# BUZZER FUNCTIONS
```

```
# =====
```

```
def buzzer_on():
```

```
    BUZZER.value(1)
```

```
def buzzer_off():
```

```
    BUZZER.value(0)
```

```
# =====
```

```
# HC-SR04 DISTANCE
```

```
# =====
```

```
def read_hcsr04():
```

```
    TRIG.value(0)
```

```
    time.sleep_us(2)
```

```
    TRIG.value(1)
```

```
    time.sleep_us(10)
```

```
TRIG.value(0)
```

```
timeout = time.ticks_us()
```

```
while ECHO.value() == 0:
```

```
    if time.ticks_diff(
```

```
        time.ticks_us(),
```

```
        timeout
```

```
    ) > 30000:
```

```
        return 400.0
```

```
start = time.ticks_us()
```

```
while ECHO.value() == 1:
```

```
    if time.ticks_diff(
```

```
        time.ticks_us(),
```

```
        start
```

```
    ) > 30000:
```

```
        return 400.0
```

```
end = time.ticks_us()
```

```
duration = time.ticks_diff(  
    end,  
    start  
)
```

```
distance = duration / 58.0
```

```
if distance < 2:  
    return 400.0
```

```
if distance > 400:  
    return 400.0
```

```
return distance
```

```
# =====
```

```
# GPS NMEA PARSER
```

```
# =====
```

```
def convert_coordinate(value, direction):
```

```
    try:
```

```
        if not value:
```

```
    return None
```

```
dot = value.find(".")
```

```
if dot < 0:
```

```
    return None
```

```
degrees_length = 2
```

```
if direction in ("E", "W"):
```

```
    degrees_length = 3
```

```
degrees = float(
```

```
    value[:degrees_length]
```

```
)
```

```
minutes = float(
```

```
    value[degrees_length:]
```

```
)
```

```
coordinate = degrees + minutes / 60.0
```

```
if direction in ("S", "W"):
```

```
    coordinate = -coordinate
```

```
return coordinate
```

```
except:  
    return None
```

```
def parse_gps():
```

```
    global gps_lat
```

```
    global gps_lon
```

```
    global gps_fix
```

```
    global gps_satellites
```

```
    while gps.any():
```

```
        try:
```

```
            line = gps.readline()
```

```
            if not line:
```

```
                continue
```

```
            line = line.decode(
```

```
                "ascii",
```

```
                "ignore"
```

```
            ).strip()
```

```
# -----
```

```
# GGA
```

```
# -----
```

```
if line.startswith("$GPGGA") or \
    line.startswith("$GNGGA"):
```

```
    data = line.split(",")
```

```
    if len(data) >= 8:
```

```
        fix_quality = data[6]
```

```
        if fix_quality != "0":
```

```
            gps_fix = True
```

```
            gps_lat = convert_coordinate(
                data[2],
                data[3]
            )
```

```
            gps_lon = convert_coordinate(
                data[4],
                data[5]
            )
```

```
else:
```

```
    gps_fix = False
```

```
try:
```

```
    gps_satellites = int(
        data[7]
    )
```

```
except:
```

```
    gps_satellites = 0
```

```
# -----
```

```
# RMC
```

```
# -----
```

```
elif line.startswith("$GPRMC") or \
    line.startswith("$GNRMC"):
```

```
    data = line.split(",")
```

```
    if len(data) >= 7:
```

```
        if data[2] == "A":
```

```
            gps_fix = True
```

```

        gps_lat = convert_coordinate(
            data[3],
            data[4]
        )

        gps_lon = convert_coordinate(
            data[5],
            data[6]
        )

    else:

        gps_fix = False

except:
    pass

# =====
# GPS STATUS STRING
# =====

def get_gps_status():

    if gps_fix and gps_lat is not None and gps_lon is not None:

```

```

    return "FIX %.6f,%.6f" % (
        gps_lat,
        gps_lon
    )

    return "NO FIX"

# =====

# CONNECT TO WIFI

# =====

wlan = network.WLAN(network.STA_IF)

wlan.active(True)

# Disable Wi-Fi power saving
try:
    wlan.config(pm=0xA11140)
except:
    pass

print()
print("=====")

```

```
print(" VEHICLE 2 - STARTING")  
print("=====")
```

```
print("Connecting to Hotspot...")
```

```
if not wlan.isconnected():
```

```
    wlan.connect(  
        SSID,  
        PASSWORD  
    )
```

```
    timeout = 20
```

```
    while not wlan.isconnected() and timeout > 0:
```

```
        print(".", end="")
```

```
        time.sleep(1)
```

```
        timeout -= 1
```

```
if wlan.isconnected():
```

```
print()
print("Wi-Fi Connected!")
```

```
print(
    "Vehicle 2 IP:",
    wlan.ifconfig()[0]
)
```

```
else:
```

```
print()
print("Wi-Fi CONNECTION FAILED")
```

```
# =====
```

```
# UDP SOCKET
```

```
# =====
```

```
sock = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM
)
```

```
sock.bind(
    ("0.0.0.0", UDP_PORT)
)
```

```
sock.setblocking(False)
```

```
print("V2V Receiver Ready")
```

```
# =====
```

```
# I2C SCAN
```

```
# =====
```

```
print()
```

```
print("I2C Devices:")
```

```
devices = i2c.scan()
```

```
for device in devices:
```

```
    print(
```

```
        hex(device)
```

```
    )
```

```
# =====
```

```
# VL53L0X INITIALIZATION
```

```
# =====
```

```
tof = None
```

```
try:
```

```
    tof = VL53L0X(i2c)
```

```
    print("VL53L0X: OK")
```

```
except Exception as e:
```

```
    print(
```

```
        "VL53L0X ERROR:",
```

```
        e
```

```
    )
```

```
# =====
```

```
# MPU6500 INITIALIZATION
```

```
# =====
```

```
mpu = None
```

```
try:
```

```
    mpu = MPU6500(i2c)
```

```
print("MPU6500: OK")
```

```
except Exception as e:
```

```
    print(  
        "MPU6500 ERROR:",  
        e  
    )
```

```
# =====
```

```
# HC-SR04 CHECK
```

```
# =====
```

```
print("HC-SR04: OK")
```

```
# =====
```

```
# GPS CHECK
```

```
# =====
```

```
print("GPS UART: OK")
```

```
# =====
```

```
# MOTOR CHECK
```

```
# =====
```

```
motor_stop()
```

```
print("Motor system: READY")
```

```
buzzer_off()
```

```
print("Buzzer: READY")
```

```
print()
```

```
print("=====")
```

```
print(" ALL SYSTEMS INITIALIZED")
```

```
print("=====")
```

```
print()
```

```
# =====
```

```
# V2V PACKET PROCESSING
```

```
# =====
```

```
def receive_v2v():
```

```
    global leader_speed
```

```
    global leader_distance
```

```
    global leader_status
```

```
global leader_gps
```

```
global last_v2v_time
```

```
try:
```

```
    data, addr = sock.recvfrom(512)
```

```
    message = data.decode().strip()
```

```
    if not message.startswith("V1"):
```

```
        return
```

```
    print()
```

```
    print("V2V:", message)
```

```
    parts = message.split(",")
```

```
    for part in parts:
```

```
        if part.startswith("SPEED:"):
```

```
            try:
```

```
                leader_speed = int(
```

```
                    part.split(":")[1]
```

```
                )
```

```
            except:
```

```
leader_speed = 0
```

```
elif part.startswith("DIST:"):
```

```
try:
```

```
    leader_distance = float(  
        part.split(":")[1]  
    )
```

```
except:
```

```
    leader_distance = 400.0
```

```
elif part.startswith("STATUS:"):
```

```
    leader_status = part.split(  
        "STATUS:"  
    )[1]
```

```
elif part.startswith("GPS:"):
```

```
    leader_gps = part.split(  
        "GPS:"  
    )[1]
```

```
last_v2v_time = time.ticks_ms()
```

```
print(
```

```
    "Leader Speed:",  
    leader_speed  
)
```

```
print(  
    "Leader Distance:",  
    leader_distance,  
    "cm"  
)
```

```
print(  
    "Leader Status:",  
    leader_status  
)
```

```
print(  
    "Leader GPS:",  
    leader_gps  
)
```

```
except:
```

```
    pass
```

```
# =====
```

```
# READ VL53L0X
```

```
# =====
```

```
def read_tof():
```

```
    if tof is None:
```

```
        return 400.0
```

```
    try:
```

```
        distance_mm = tof.range
```

```
        distance_cm = distance_mm / 10.0
```

```
        if distance_cm <= 0:
```

```
            return 400.0
```

```
        if distance_cm > 400:
```

```
            return 400.0
```

```
        return distance_cm
```

```
    except:
```

```
        return 400.0
```

```
# =====
```

```
# READ MPU6500
```

```
# =====
```

```
def read_mpu():
```

```
    if mpu is None:
```

```
        return (
```

```
            0,
```

```
            0,
```

```
            0,
```

```
            0,
```

```
            0,
```

```
            0
```

```
        )
```

```
    try:
```

```
        accel = mpu.acceleration
```

```
        gyro = mpu.gyro
```

```
        ax = accel[0]
```

```
        ay = accel[1]
```

```
az = accel[2]
```

```
gx = gyro[0]
```

```
gy = gyro[1]
```

```
gz = gyro[2]
```

```
return (
```

```
    ax,
```

```
    ay,
```

```
    az,
```

```
    gx,
```

```
    gy,
```

```
    gz
```

```
)
```

```
except:
```

```
return (
```

```
    0,
```

```
    0,
```

```
    0,
```

```
    0,
```

```
    0,
```

```
    0
```

```
)
```

```
# =====
```

```
# SAFETY CONTROL
```

```
# =====
```

```
def safety_control(
```

```
    local_distance
```

```
):
```

```
    global leader_speed
```

```
    global leader_status
```

```
    global last_v2v_time
```

```
# -----
```

```
# 1. V2V TIMEOUT
```

```
# -----
```

```
if time.ticks_diff(
```

```
    time.ticks_ms(),
```

```
    last_v2v_time
```

```
) > 2000:
```

```
    motor_stop()
```

```
    buzzer_on()
```

```
return (  
    0,  
    "V2V LOST"  
)
```

```
# -----
```

```
# 2. LOCAL EMERGENCY OBSTACLE
```

```
# -----
```

```
if local_distance <= 15:
```

```
    motor_stop()
```

```
    buzzer_on()
```

```
return (  
    0,  
    "LOCAL EMERGENCY STOP"  
)
```

```
# -----
```

```
# 3. LOCAL WARNING
```

```
# -----
```

```
if local_distance <= 30:
```

```
    motor_forward(30)
```

```
    buzzer_on()
```

```
    return (
```

```
        30,
```

```
        "LOCAL WARNING"
```

```
    )
```

```
# -----
```

```
# 4. LEADER EMERGENCY
```

```
# -----
```

```
if "EMERGENCY" in leader_status:
```

```
    motor_stop()
```

```
    buzzer_on()
```

```
    return (
```

```
        0,
```

```
        "LEADER EMERGENCY STOP"
```

```
    )
```

```
# -----
```

```
# 5. LEADER WARNING
```

```
# -----
```

```
if "WARNING" in leader_status:
```

```
    motor_forward(30)
```

```
    buzzer_on()
```

```
    return (
```

```
        30,
```

```
        "LEADER WARNING"
```

```
    )
```

```
# -----
```

```
# 6. LEADER SAFE
```

```
# -----
```

```
if "SAFE" in leader_status:
```

```
    buzzer_off()
```

```
speed = leader_speed
```

```
if speed < 0:
```

```
    speed = 0
```

```
if speed > 75:
```

```
    speed = 75
```

```
motor_forward(speed)
```

```
return (
```

```
    speed,
```

```
    "FOLLOWING LEADER"
```

```
)
```

```
# -----
```

```
# 7. UNKNOWN STATE
```

```
# -----
```

```
motor_stop()
```

```
buzzer_on()
```

```
return (
```

```
    0,
```

```
"UNKNOWN - STOP"
```

```
)
```

```
# =====
```

```
# MAIN LOOP
```

```
# =====
```

```
last_sensor_print = time.ticks_ms()
```

```
while True:
```

```
# -----
```

```
# Receive V2V
```

```
# -----
```

```
receive_v2v()
```

```
# -----
```

```
# Read local sensors
```

```
# -----
```

```
tof_distance = read_tof()
```

```
ultrasonic_distance = read_hcsr04()
```

```
# -----
```

```
# Choose closest obstacle
```

```
# -----
```

```
local_distance = min(  
    tof_distance,  
    ultrasonic_distance  
)
```

```
# -----
```

```
# Read MPU6500
```

```
# -----
```

```
(  
    ax,  
    ay,  
    az,  
    gx,  
    gy,  
    gz  
) = read_mpu()
```

```
# -----
```

```
# Read GPS
```

```
# -----
```

```
parse_gps()
```

```
# -----
```

```
# Safety control
```

```
# -----
```

```
actual_speed, status = safety_control(
```

```
    local_distance
```

```
)
```

```
# -----
```

```
# Display sensor information
```

```
# -----
```

```
if time.ticks_diff(
```

```
    time.ticks_ms(),
```

```
    last_sensor_print
```

```
) >= 1000:
```

```
    last_sensor_print = time.ticks_ms()
```

```
print()
print("-----")
```

```
print(
    "V2 VL53L0X:",
    round(tof_distance, 1),
    "cm"
)
```

```
print(
    "V2 HC-SR04:",
    round(ultrasonic_distance, 1),
    "cm"
)
```

```
print(
    "V2 Closest:",
    round(local_distance, 1),
    "cm"
)
```

```
print(
    "Speed:",
    actual_speed,
    "%"
```

```
)
```

```
print(  
    "Status:",  
    status  
)
```

```
print(  
    "ACC:",  
    round(ax, 2),  
    round(ay, 2),  
    round(az, 2)  
)
```

```
print(  
    "GYRO:",  
    round(gx, 2),  
    round(gy, 2),  
    round(gz, 2)  
)
```

```
print(  
    "GPS:",  
    get_gps_status()  
)
```

```
print(  
    "Satellites:",  
    gps_satellites  
)
```

```
print(  
    "Leader:",  
    leader_status,  
    "|",  
    leader_speed,  
    "%"  
)
```

```
print("-----")
```

```
# -----  
# Garbage collection  
# -----
```

```
gc.collect()
```

```
time.sleep_ms(50)
```